

Kartavya Mandora_100876605_Final Project_Code_S3

May 3, 2026

```
[1]: import os
import zipfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy.io import loadmat
```

```
[2]: zip_path = "BCICIV_4_mat.zip"
extract_path = "BCICIV_4_data"

print("Zip path:", zip_path)
print("Extract folder:", extract_path)
```

Zip path: BCICIV_4_mat.zip
Extract folder: BCICIV_4_data

```
[3]: if not os.path.exists(extract_path):
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_path)
    print("Dataset extracted successfully.")
else:
    print("Dataset already extracted.")
```

Dataset already extracted.

```
[4]: for root, dirs, files in os.walk(extract_path):
    print("Folder:", root)
    for file in files:
        print("  ", file)
```

Folder: BCICIV_4_data
sub1_comp.mat
sub2_comp.mat
sub3_comp.mat

```
[5]: file_path = os.path.join(extract_path, "sub3_comp.mat")
data = loadmat(file_path)
```

```
print("Loaded file:", file_path)
```

Loaded file: BCICIV_4_data\sub3_comp.mat

```
[6]: print("Keys in the .mat file:")
      print(data.keys())
```

Keys in the .mat file:

```
dict_keys(['__header__', '__version__', '__globals__', 'train_data',
'test_data', 'train_dg'])
```

```
[7]: train_data = data["train_data"]
      train_dg = data["train_dg"]
      test_data = data["test_data"]

      print("train_data shape:", train_data.shape)
      print("train_dg shape:", train_dg.shape)
      print("test_data shape:", test_data.shape)
```

train_data shape: (400000, 64)

train_dg shape: (400000, 5)

test_data shape: (200000, 64)

```
[8]: n_samples, n_channels = train_data.shape
      n_targets = train_dg.shape[1]

      print(f"Number of training samples: {n_samples}")
      print(f"Number of neural channels: {n_channels}")
      print(f"Number of target outputs (fingers): {n_targets}")
```

Number of training samples: 400000

Number of neural channels: 64

Number of target outputs (fingers): 5

```
[9]: print("First 5 rows of train_data:")
      print(train_data[:5])

      print("\nFirst 5 rows of train_dg:")
      print(train_dg[:5])
```

First 5 rows of train_data:

```
[[ 1392  2132  1811   -90  2567  -493  1254 -1096  -667   426   -46  1365
   -475 -1160 -2297  -524   350   638 -3016   961  2192  -578  -990 -1400
   1287  -808  1589  1597 -6537   -82 -1108 -5447   813   973   356   848
   3780  1991  1527  2192  2431  2915   188  -331  1748  3701 -1392   987
   1732  1460  3443 -2319  1496  3209  1818  2280   553  4157 -1699  -475
   1357  -705   -15 -1290]
 [  842  1850  1461  -350  2478  -679  1124 -1289 -1089   -19  -373   722
  -663 -1041 -2427  -860   156   455 -3433   787  1955  -900 -1162 -1683
   691 -1216  1348  1027 -6792  -503 -1318 -5742   585   661   119   726]
```

```

3770 1716 1223 1948 2150 2584 90 -359 1548 3574 -1405 768
1616 1268 3218 -2494 1262 3028 1561 2097 310 3730 -1913 -707
1163 -897 -385 -1558]
[ 582 1912 1505 -237 2689 -532 1366 -961 -1035 3 -339 386
-477 -609 -2226 -699 421 609 -3207 1031 2047 -846 -1129 -1568
447 -1265 1729 764 -6603 -452 -1091 -5725 694 713 285 980
4161 1754 1361 2040 2148 2572 360 -55 1667 3882 -967 949
1878 1419 3456 -2199 1545 3178 1705 2444 501 3661 -2251 -583
1336 -549 -391 -1414]
[ 183 1679 1387 -458 2593 -688 1387 -832 -1125 -255 -494 -87
-601 -545 -2224 -753 364 455 -3175 1078 1883 -997 -1233 -1701
9 -1521 1836 322 -6675 -683 -1076 -5926 489 559 200 1026
4317 1543 1335 1768 2090 2200 355 28 1571 3918 -805 972
1913 1376 3343 -2164 1526 3117 1559 2600 387 3425 -2537 -700
1108 -488 -596 -1576]
[ -253 1145 1039 -1032 2176 -1130 1125 -1003 -1442 -864 -860 -646
-1080 -872 -2437 -1087 -59 28 -3567 832 1439 -1390 -1444 -2063
-581 -2056 1476 -271 -7065 -1204 -1357 -6336 9 182 -179 798
4122 1029 1018 1179 1970 1470 0 -142 1248 3622 -960 734
1661 1105 2778 -2471 1108 2782 1085 2396 -109 2991 -2317 -1048
373 -786 -1008 -2088]]

```

First 5 rows of train_dg:

```

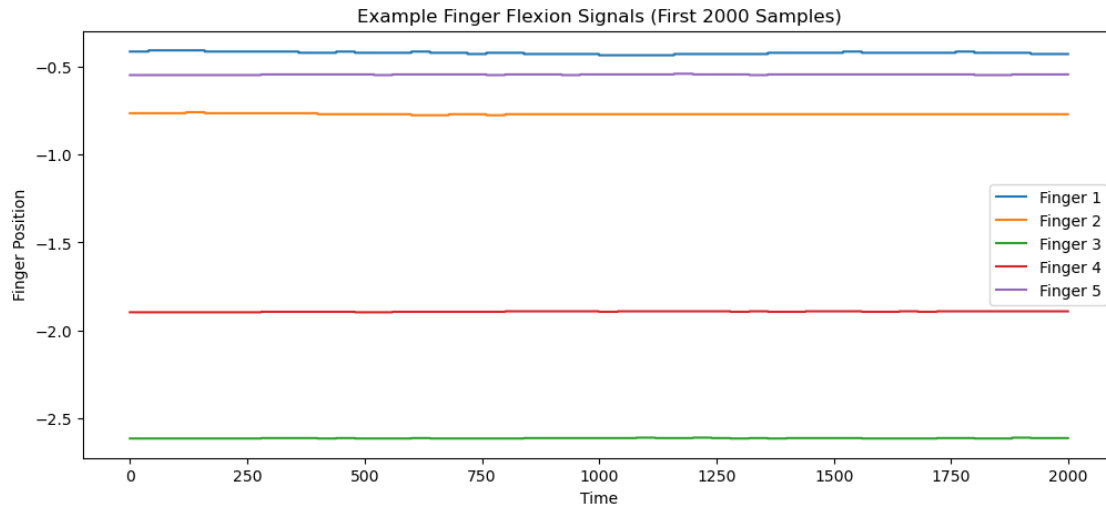
[[-0.41384039 -0.76443916 -2.61538669 -1.89710032 -0.54750119]
 [-0.41384039 -0.76443916 -2.61538669 -1.89710032 -0.54750119]
 [-0.41384039 -0.76443916 -2.61538669 -1.89710032 -0.54750119]
 [-0.41384039 -0.76443916 -2.61538669 -1.89710032 -0.54750119]
 [-0.41384039 -0.76443916 -2.61538669 -1.89710032 -0.54750119]]

```

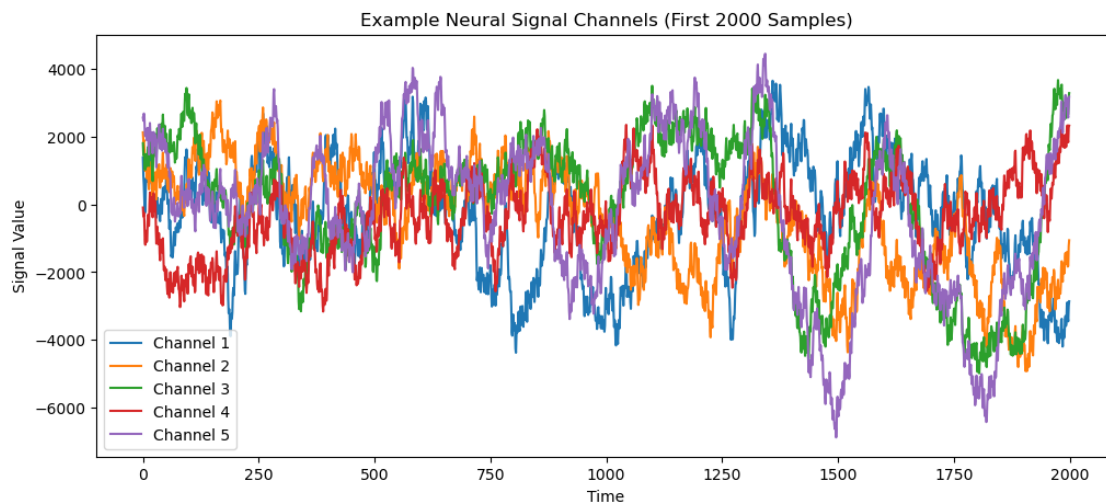
```

[10]: plt.figure(figsize=(12, 5))
      for i in range(train_dg.shape[1]):
          plt.plot(train_dg[:2000, i], label=f'Finger {i+1}')
      plt.title("Example Finger Flexion Signals (First 2000 Samples)")
      plt.xlabel("Time")
      plt.ylabel("Finger Position")
      plt.legend()
      plt.show()

```



```
[11]: plt.figure(figsize=(12, 5))
      for i in range(min(5, train_data.shape[1])):
          plt.plot(train_data[:2000, i], label=f'Channel {i+1}')
      plt.title("Example Neural Signal Channels (First 2000 Samples)")
      plt.xlabel("Time")
      plt.ylabel("Signal Value")
      plt.legend()
      plt.show()
```



```
[12]: print("Missing values in train_data:", np.isnan(train_data).sum())
      print("Missing values in train_dg:", np.isnan(train_dg).sum())
      print("Missing values in test_data:", np.isnan(test_data).sum())
```

Missing values in train_data: 0
Missing values in train_dg: 0
Missing values in test_data: 0

```
[13]: X = train_data
      y = train_dg

      print("Input X shape:", X.shape)
      print("Target y shape:", y.shape)
```

Input X shape: (400000, 64)
Target y shape: (400000, 5)

```
[14]: from sklearn.model_selection import train_test_split

      X_train, X_val, y_train, y_val = train_test_split(
          X, y, test_size=0.2, random_state=42, shuffle=True
      )

      print("X_train:", X_train.shape)
      print("X_val:", X_val.shape)
```

X_train: (320000, 64)
X_val: (80000, 64)

```
[15]: from sklearn.preprocessing import StandardScaler

      scaler = StandardScaler()

      X_train_scaled = scaler.fit_transform(X_train)
      X_val_scaled = scaler.transform(X_val)

      print("Scaling complete.")
```

Scaling complete.

```
[16]: from sklearn.linear_model import LinearRegression
      from sklearn.multioutput import MultiOutputRegressor

      baseline_model = MultiOutputRegressor(LinearRegression())
      baseline_model.fit(X_train_scaled, y_train)

      print("Wiener Filter (Linear Regression) trained.")
```

Wiener Filter (Linear Regression) trained.

```
[17]: from sklearn.metrics import r2_score

      y_pred_baseline = baseline_model.predict(X_val_scaled)
```

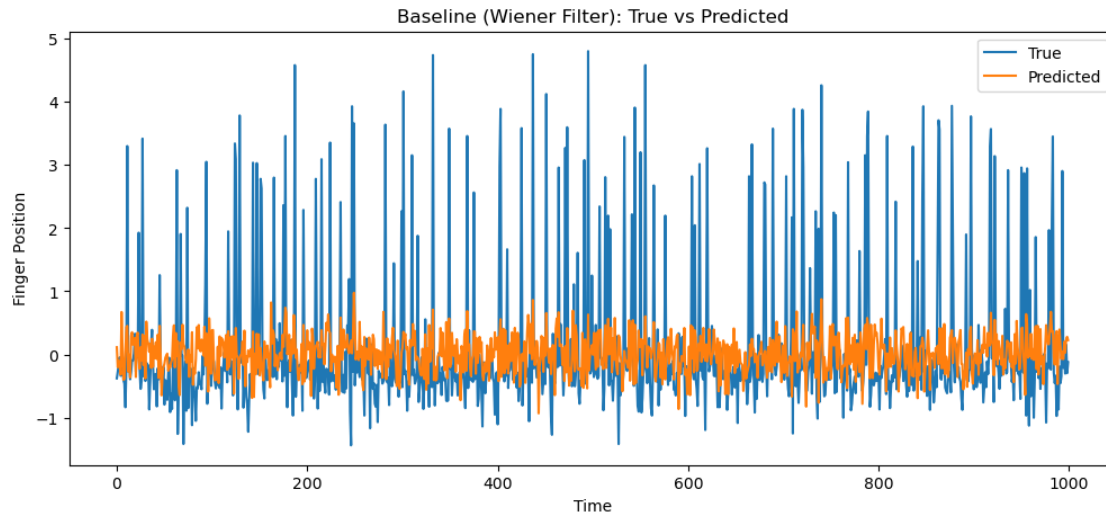
```
baseline_r2 = r2_score(y_val, y_pred_baseline, multioutput='uniform_average')

print("Baseline Wiener Filter R2 score:", baseline_r2)
```

Baseline Wiener Filter R² score: 0.08489723372396199

```
[18]: finger_idx = 0 # choose one finger

plt.figure(figsize=(12, 5))
plt.plot(y_val[:1000, finger_idx], label="True")
plt.plot(y_pred_baseline[:1000, finger_idx], label="Predicted")
plt.title("Baseline (Wiener Filter): True vs Predicted")
plt.xlabel("Time")
plt.ylabel("Finger Position")
plt.legend()
plt.show()
```



```
[19]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

```
[20]: nn_model = Sequential([
    Dense(128, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dropout(0.2),

    Dense(64, activation='relu'),
    Dropout(0.2),

    Dense(y_train.shape[1], activation='linear')
```

```

])

nn_model.compile(
    optimizer='adam',
    loss='mse'
)

nn_model.summary()

```

C:\Users\karta\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:95:
 UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
 using Sequential models, prefer using an `Input(shape)` object as the first
 layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type) ↳ Param #	Output Shape	
dense (Dense) ↳ 8,320	(None, 128)	↳
dropout (Dropout) ↳ 0	(None, 128)	↳
dense_1 (Dense) ↳ 8,256	(None, 64)	↳
dropout_1 (Dropout) ↳ 0	(None, 64)	↳
dense_2 (Dense) ↳ 325	(None, 5)	↳

Total params: 16,901 (66.02 KB)

Trainable params: 16,901 (66.02 KB)

Non-trainable params: 0 (0.00 B)

```
[21]: early_stop = EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True
    )

    history = nn_model.fit(
        X_train_scaled, y_train,
        validation_data=(X_val_scaled, y_val),
        epochs=30,
        batch_size=256,
        callbacks=[early_stop],
        verbose=1
    )
```

```
Epoch 1/30
1250/1250          2s 1ms/step -
loss: 0.9051 - val_loss: 0.7376
Epoch 2/30
1250/1250          2s 1ms/step -
loss: 0.7530 - val_loss: 0.6490
Epoch 3/30
1250/1250          2s 1ms/step -
loss: 0.6929 - val_loss: 0.5977
Epoch 4/30
1250/1250          2s 1ms/step -
loss: 0.6607 - val_loss: 0.5664
Epoch 5/30
1250/1250          2s 1ms/step -
loss: 0.6376 - val_loss: 0.5472
Epoch 6/30
1250/1250          2s 1ms/step -
loss: 0.6224 - val_loss: 0.5326
Epoch 7/30
1250/1250          2s 1ms/step -
loss: 0.6116 - val_loss: 0.5146
Epoch 8/30
1250/1250          2s 1ms/step -
loss: 0.6028 - val_loss: 0.5084
Epoch 9/30
1250/1250          2s 1ms/step -
loss: 0.5959 - val_loss: 0.4975
Epoch 10/30
1250/1250          2s 1ms/step -
loss: 0.5903 - val_loss: 0.4971
Epoch 11/30
1250/1250          2s 1ms/step -
loss: 0.5859 - val_loss: 0.4873
```


Epoch 12/30
1250/1250 2s 1ms/step -
loss: 0.5806 - val_loss: 0.4850
Epoch 13/30
1250/1250 2s 1ms/step -
loss: 0.5775 - val_loss: 0.4819
Epoch 14/30
1250/1250 1s 1ms/step -
loss: 0.5743 - val_loss: 0.4756
Epoch 15/30
1250/1250 1s 1ms/step -
loss: 0.5702 - val_loss: 0.4745
Epoch 16/30
1250/1250 2s 1ms/step -
loss: 0.5681 - val_loss: 0.4666
Epoch 17/30
1250/1250 1s 1ms/step -
loss: 0.5655 - val_loss: 0.4692
Epoch 18/30
1250/1250 1s 1ms/step -
loss: 0.5642 - val_loss: 0.4647
Epoch 19/30
1250/1250 1s 1ms/step -
loss: 0.5617 - val_loss: 0.4580
Epoch 20/30
1250/1250 1s 1ms/step -
loss: 0.5587 - val_loss: 0.4564
Epoch 21/30
1250/1250 1s 1ms/step -
loss: 0.5590 - val_loss: 0.4567
Epoch 22/30
1250/1250 2s 1ms/step -
loss: 0.5573 - val_loss: 0.4584
Epoch 23/30
1250/1250 1s 1ms/step -
loss: 0.5545 - val_loss: 0.4551
Epoch 24/30
1250/1250 2s 1ms/step -
loss: 0.5538 - val_loss: 0.4496
Epoch 25/30
1250/1250 1s 1ms/step -
loss: 0.5516 - val_loss: 0.4512
Epoch 26/30
1250/1250 1s 1ms/step -
loss: 0.5521 - val_loss: 0.4525
Epoch 27/30
1250/1250 1s 1ms/step -
loss: 0.5504 - val_loss: 0.4470

```
Epoch 28/30
1250/1250          1s 1ms/step -
loss: 0.5486 - val_loss: 0.4471
Epoch 29/30
1250/1250          1s 1ms/step -
loss: 0.5493 - val_loss: 0.4490
Epoch 30/30
1250/1250          1s 1ms/step -
loss: 0.5467 - val_loss: 0.4451
```

```
[22]: y_pred_nn_original = nn_model.predict(X_val_scaled)

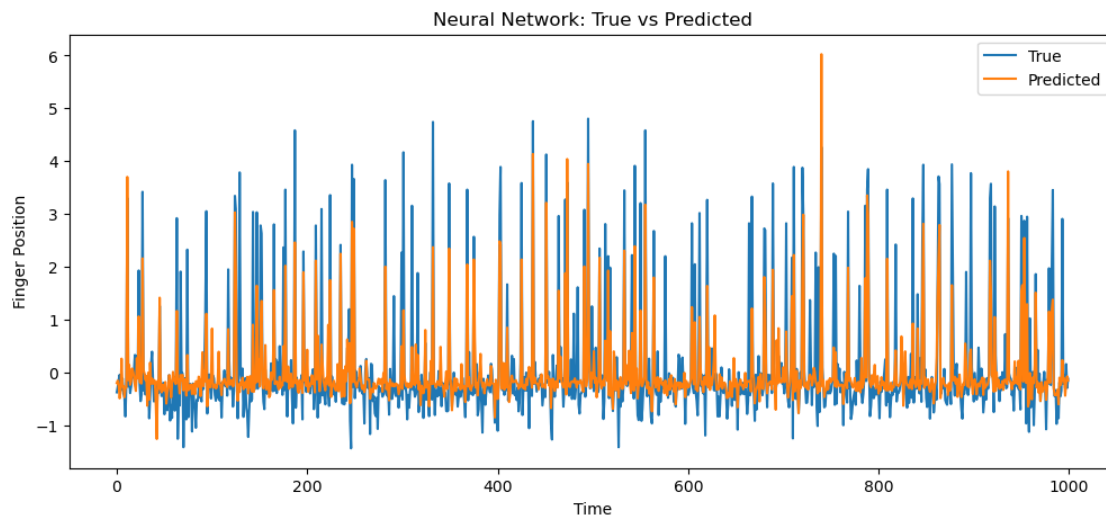
nn_r2 = r2_score(y_val, y_pred_nn_original, multioutput='uniform_average')

print("Neural Network R2 score:", nn_r2)
```

```
2500/2500          1s 401us/step
Neural Network R2 score: 0.5925782128106782
```

```
[23]: finger_idx = 0

plt.figure(figsize=(12, 5))
plt.plot(y_val[:1000, finger_idx], label="True")
plt.plot(y_pred_nn_original[:1000, finger_idx], label="Predicted")
plt.title("Neural Network: True vs Predicted")
plt.xlabel("Time")
plt.ylabel("Finger Position")
plt.legend()
plt.show()
```



```
[24]: def apply_channel_dropout(X, drop_fraction=0.3, random_state=42):
        np.random.seed(random_state)
        X_copy = X.copy()
        n_channels = X_copy.shape[1]
        n_drop = int(n_channels * drop_fraction)
        dropped_channels = np.random.choice(n_channels, n_drop, replace=False)
        X_copy[:, dropped_channels] = 0
        return X_copy, dropped_channels
```

```
[25]: X_val_drop30, dropped_channels = apply_channel_dropout(X_val_scaled,
        ↪drop_fraction=0.3)

print("Dropped channel indices:", dropped_channels)
print("Number of dropped channels:", len(dropped_channels))
```

Dropped channel indices: [52 58 0 44 5 36 16 12 25 61 56 9 40 13 34 50 3 17 49]

Number of dropped channels: 19

```
[26]: y_pred_baseline_drop = baseline_model.predict(X_val_drop30)
baseline_drop_r2 = r2_score(y_val, y_pred_baseline_drop,
        ↪multioutput='uniform_average')

print("Wiener Filter R² with 30% channel dropout:", baseline_drop_r2)
```

Wiener Filter R² with 30% channel dropout: 0.05916784769832928

```
[27]: y_pred_nn_drop = nn_model.predict(X_val_drop30)
nn_drop_r2 = r2_score(y_val, y_pred_nn_drop, multioutput='uniform_average')

print("Neural Network R² with 30% channel dropout:", nn_drop_r2)
```

2500/2500 1s 388us/step

Neural Network R² with 30% channel dropout: 0.2584169839803777

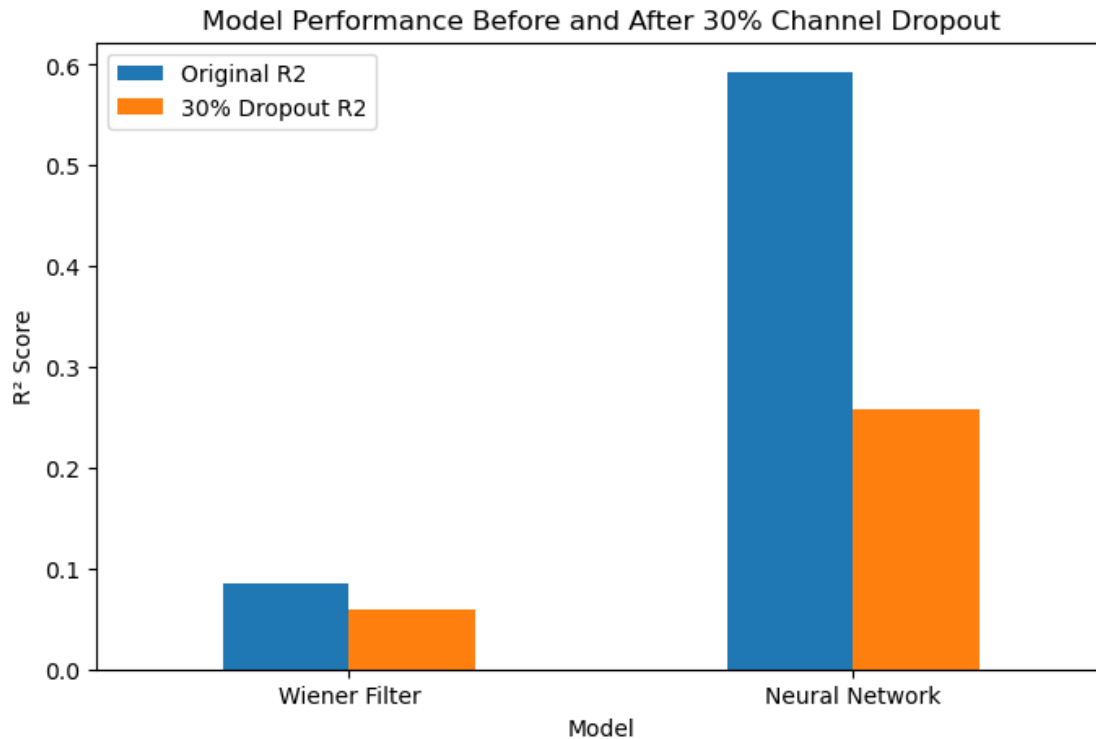
```
[28]: results = pd.DataFrame({
        "Model": ["Wiener Filter", "Neural Network"],
        "Original R2": [baseline_r2, nn_r2],
        "30% Dropout R2": [baseline_drop_r2, nn_drop_r2]
    })

print(results)
```

	Model	Original R2	30% Dropout R2
0	Wiener Filter	0.084897	0.059168
1	Neural Network	0.592578	0.258417

```
[29]: results.set_index("Model").plot(kind="bar", figsize=(8, 5))
plt.title("Model Performance Before and After 30% Channel Dropout")
```

```
plt.ylabel("R2 Score")
plt.xticks(rotation=0)
plt.show()
```



```
[30]: dropout_levels = [0.0, 0.1, 0.3, 0.5, 0.7]
```

```
[31]: dropout_results = []

for drop in dropout_levels:
    if drop == 0.0:
        X_test_drop = X_val_scaled
        dropped_channels = []
    else:
        X_test_drop, dropped_channels = apply_channel_dropout(
            X_val_scaled,
            drop_fraction=drop,
            random_state=42
        )

    y_pred_wiener = baseline_model.predict(X_test_drop)
    y_pred_nn_drop_loop = nn_model.predict(X_test_drop)

    wiener_r2 = r2_score(y_val, y_pred_wiener, multioutput='uniform_average')
```

```

nn_r2_loop = r2_score(y_val, y_pred_nn_drop_loop,
↳multioutput='uniform_average')

dropout_results.append({
    "Dropout Level": f"{int(drop * 100)}%",
    "Wiener Filter R2": wiener_r2,
    "Neural Network R2": nn_r2_loop,
    "Dropped Channels": len(dropped_channels)
})

dropout_results_df = pd.DataFrame(dropout_results)
dropout_results_df

```

```

2500/2500      1s 461us/step
2500/2500      1s 405us/step
2500/2500      1s 398us/step
2500/2500      1s 402us/step
2500/2500      1s 401us/step

```

```

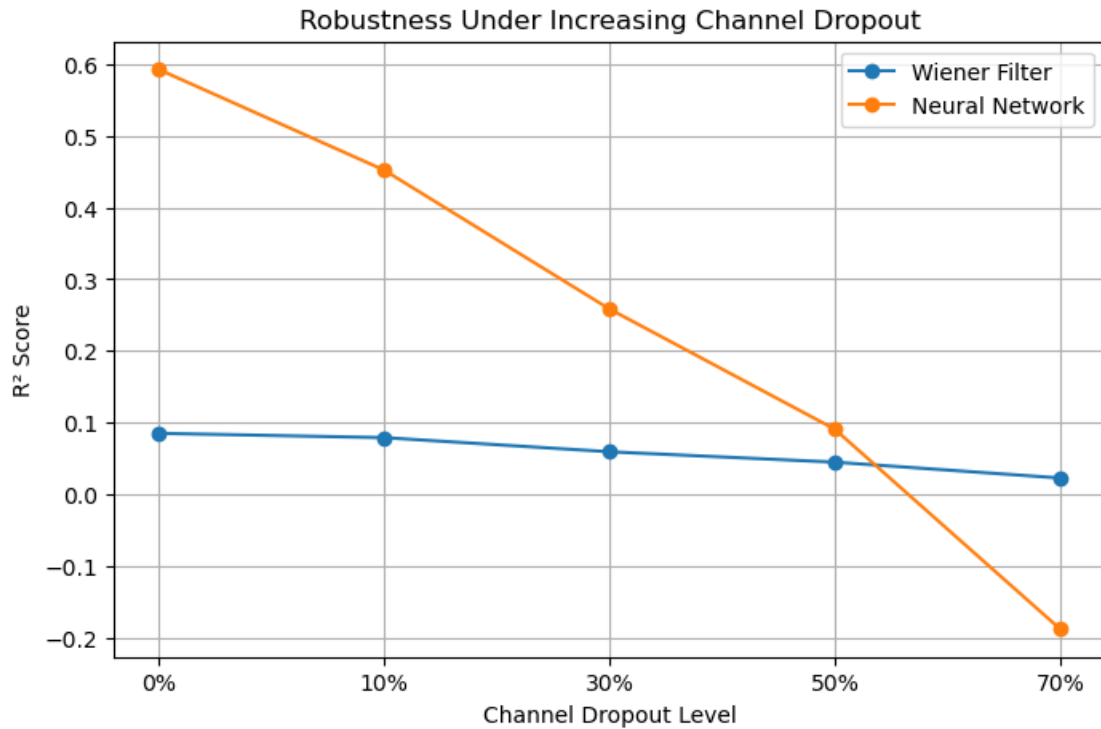
[31]: Dropout Level  Wiener Filter R2  Neural Network R2  Dropped Channels
0           0%           0.084897           0.592578           0
1          10%           0.078869           0.452148           6
2          30%           0.059168           0.258417          19
3          50%           0.044663           0.090721          32
4          70%           0.022564          -0.187991          44

```

```

[32]: plt.figure(figsize=(8, 5))
plt.plot(dropout_results_df["Dropout Level"], dropout_results_df["Wiener Filter_
↳R2"], marker='o', label="Wiener Filter")
plt.plot(dropout_results_df["Dropout Level"], dropout_results_df["Neural_
↳Network R2"], marker='o', label="Neural Network")
plt.title("Robustness Under Increasing Channel Dropout")
plt.xlabel("Channel Dropout Level")
plt.ylabel("R2 Score")
plt.legend()
plt.grid(True)
plt.show()

```



```
[33]: finger_r2_results = []

for i in range(y_val.shape[1]):
    finger_r2_results.append({
        "Finger": f"Finger {i+1}",
        "Wiener Filter R2": r2_score(y_val[:, i], y_pred_baseline[:, i]),
        "Neural Network R2": r2_score(y_val[:, i], y_pred_nn_original[:, i])
    })

finger_r2_df = pd.DataFrame(finger_r2_results)
finger_r2_df
```

```
[33]:
```

	Finger	Wiener Filter R2	Neural Network R2
0	Finger 1	0.080129	0.674290
1	Finger 2	0.127952	0.591025
2	Finger 3	0.065546	0.493195
3	Finger 4	0.095249	0.579901
4	Finger 5	0.055610	0.624481

```
[34]: finger_r2_df.set_index("Finger").plot(kind="bar", figsize=(9, 5))
plt.title("Per-Finger Decoding Performance")
plt.ylabel("R² Score")
plt.xticks(rotation=0)
```

```
plt.show()
```

